國 立 成 功 大 學

電 腦 與 通 信 工 程 研 究 所

碩 士 論 文

類別不平衡下的入侵偵測二分類區域自訓練框架

A Regional Self-Training Framework for Binary Classification of Intrusion Detection under Class Imbalance

研究生： 許承瀚 Student: Cheng-Han Hsu

指導教授：詹寶珠 Advisor: Pau-Choo Chung

指導教授：謝錫堃 Advisor: Ce-Kuen Shieh

Institute of Computer and Communication Engineering

National Cheng Kung University

Tainan, Taiwan

July 2025

中華民國 一一四年七月

國立成功大學

National Cheng Kung University

碩士論文合格證明書

Certificate of Approval for Master's Thesis

題目；類別不平衡下的入侵偵測二分類區域自訓練框架

Title：A Regional Self-Training Framework for Binary Classification of Intrusion Detection under Class Imbalance

研究生：許承瀚

本論文業經審查及口試合格特此證明

This is to certify that the Master's Thesis of HSU,CHENG-HAN

論文考試委員：

has passed the oral defense under the decision of the following committee members

謝錫堃

黃文祥 曾黎明 張志穆 詹寶珠

指導教授 Advisor(s)：詹寶珠 謝錫堃

單位主管 Chair/Director：陳昭羽

（單位主管是否簽章授權由各院、系（所、學位學程）自訂）

(The certificate must be signed by the committee members and advisor(s). Each department/graduate institute/degree program can determine whether the chair/director also needs to sign.)

2025/7/25

類別不平衡下的入侵偵測二分類區域自訓練框架

許承瀚* 詹寶珠** 謝錫堃**

國立成功大學

電腦與通信工程研究所

摘要

隨著網路攻擊手法日益複雜與隱匿化，入侵偵測系統扮演維護網路安全的重要防線。即時流量分析與監控能協助發現網路中的異常行為，但要進一步準確判別異常類型，仍必須仰賴大量且具代表性的標記資料。當標記樣本不足且資料分布失衡時，傳統監督式學習方法的效能即大幅受限。此外，標註過程需仰賴複雜且耗費人力的流程，難以取得充足且具代表性的樣本；資料分布失衡時，模型易產生偏倚，且訓練過程也變得不穩定。有鑑於此，本文提出一種區域自訓練框架，同時結合 基於 HDBSCAN 與 KNN check 的密度感知偽標籤生成，以及 針對少數類別的區域式過採樣，以同時提升偽標籤品質與少數類別覆蓋率。實驗以 UNSW-NB15 與 CICIDS2017 資料集進行評估，在僅使用 1% 標註樣本的情況下，本方法在 UNSW-NB15 上的整體 F1-score 由傳統自訓練法的 93.6% 提升至 96.8%（+3.2 個百分點），少數類別 Recall 則由 94.5% 提升至 98.0%（+3.5 個百分點）。在 CICIDS2017 上，本方法的整體 F1-score 由 92.7% 提升至 96.4%（+3.7 個百分點），少數類別 Recall 則由 93.9% 提升至 97.8%（+3.9 個百分點）。結果顯示，本研究在標註資料稀缺與類別不均的情境下，能穩定提升整體分類效能，並顯著強化對少數類別異常樣本的識別能力。

關鍵字：入侵偵測、自訓練、類別不平衡、深度學習

*作者　**指導教授

A Regional Self-Training Framework for Binary Classification of Intrusion Detection under Class Imbalance

Cheng-Han Hsu * Pau-Choo Chung** Ce-Kuen Shieh**

Institute of Computer and Communication Engineering

National Cheng Kung University

Tainan, Taiwan

Abstract

As network attack techniques become increasingly sophisticated and stealthy, intrusion detection systems (IDS) play a critical role in maintaining cybersecurity. Real-time traffic analysis and monitoring can help uncover anomalous behaviors on the network, but accurately classifying these anomalies still requires large volumes of representative labeled data. When labeled samples are scarce and class distributions are imbalanced, the performance of traditional supervised learning methods is severely limited. Furthermore, the labeling process relies on complex and labor-intensive workflows, making it difficult to obtain sufficient and representative samples. Under imbalanced conditions, models are also prone to bias, and training becomes unstable.

To address these challenges, this paper proposes a Regional Self-Training framework that integrates density-aware pseudo-labeling—using HDBSCAN combined with KNN check—with regional oversampling for minority classes, thereby simultaneously enhancing pseudo-label quality and improving minority-class coverage.

Experiments were conducted on the UNSW-NB15 and CICIDS2017 datasets with only 1% labeled samples. On UNSW-NB15, our method improved the overall F1-score from 93.6% with standard self-training to 96.8% (+3.2 percentage points), while the recall of minority-class attacks increased from 94.5% to 98.0% (+3.5 percentage points). On CICIDS2017, the overall F1-score rose from 92.7% to 96.4% (+3.7 percentage points), and minority-class recall improved from 93.9% to

97.8% (+3.9 percentage points). These results demonstrate that the proposed framework can reliably enhance overall classification performance and significantly strengthen the detection of minority-class anomalies under conditions of scarce labeled data and class imbalance.

Keywords: intrusion detection, self-training, class imbalance, Deep Learning

*Author **Advisor

誌　謝

即將離開校園之際，這段研究所生活的點滴仍歷歷在目。回首這段歷程，滿懷感激，謹向在此期間給予我支持與指導的所有人致以最誠摯的敬意。首先，要特別感謝我的指導教授謝錫堃老師，在我的研究歷程中給予莫大的幫助。謝老師不僅在網路安全等專業領域為我提供寶貴建議，亦在學術研究原則與研究方向上悉心引領。您的專業知識與耐心指點，讓我在學術上獲益匪淺，並激勵我在求學與研究中持續追求卓越。同時，也感謝詹寶珠老師，特別是在論文完成階段，為我提供結構與內容上的指導，協助我優化與完善整體品質。您的悉心協助與專業見解，對本研究的完成起到了關鍵的推動作用。此外，衷心感謝張志標學長、王俊又學長，以及博士班的羅大郡學長，在我假日時於學術討論中慷慨分享經驗與看法，並實際參與實驗設計與資料處理等面向，讓我的研究能順利推進。感謝一路相伴的實驗室同學與學長們，當我面臨疑惑時，你們總能提供有效建議與不同視角，讓我們共同克服各種挑戰。再者，感謝曾黎明老師、黃文祥老師，以及張志標老師在論文過程中提出的中肯建議，這些意見成為我完成研究的重要養分，並促使本研究更加完善。最後，衷心感謝家人在研究所期間給予的精神鼓勵與陪伴，使我能專注於研究並順利完成論文。

Content

Chapter1: Introduction **1**
Chapter2: Background and Related Works **3**
2.1 Background 3
2.1.1 Self-Training *3*
2.1.2 Pseudo Labeling *4*
2.1.3 HDBSCAN *5*
2.1.4 SMOTE *7*
2.2 Related Works 9
2.2.1 Limitations of Traditional IDS *9*
2.2.2 Semi-Supervised Learning in IDS *9*
2.2.3 Self-training under Class Imbalance *10*
Chapter3: Methodology **12**
3.1 System Overview 12
3.2 Density-aware Pseudo-labeling 13
3.2.1 Clustering by HDBSCAN *13*
3.2.2 Pseudo-label Assignment *14*
3.2.3 Pseudo-label Validation via KNN check *15*
3.3 Regional Oversampling 16
3.4 Algorithm 18
Chapter 4: Experiment **19**
4.1 Experiment Setup 19
4.1.1 Datasets *19*
4.1.2 Data Preprocessing *21*
4.1.3 Environment *21*
4.1.4 Base Model Structure *21*
4.1.5 Evaluation Criteria *22*
4.2 Experiment Result 23
4.2.1 Experiment 1: Binary Classification on UNSW-NB15 ..... *23*
4.2.2 Experiment 2: Ablation Study on UNSW-NB15 ..... *24*
4.2.3 Experiment 3: Comparison of Different SMOTE Variants *26*
4.2.4 Experiment 4: Binary Classification on CICIDS2017 ..... *28*
Chapter 5: Conclusion **29**
Chapter 6: Future Work **30**
Chapter 7: References **31**

List of Figures

Figure 1 Flowchart of the Self-Training Process 4
Figure 2. Euclidean space vs. λ-space in HDBSCAN [23] 6
Figure 3. Performance of clustering algorithms [24] 7
Figure 4. SMOTE Process [15] 8
Figure 5. Regional Self-Training Framework System Overview 12
Figure 6.HDBSCAN Clustering Results 13
Figure 7. HDBSCAN Cluster Categories by Label Composition 14
Figure 8. Pseudo-label refinement through KNN check 15
Figure 9.In-Cluster SMOTE for Minority Class Augmentation in Medium-Purity Regions 16
Figure 10. Proportional In-Cluster SMOTE in High-Purity Minority Clusters 17
Figure 11. Algorithm for Regional self-training Framework 18
Figure 12. Normal vs. Malicious Traffic Proportions in UNSW-NB15 and CICIDS2017 20
Figure 13. Base Model Architecture 22
Figure 14. Ablation Study Results (F1-score) 25
Figure 15. SMOTE Variant Comparison (F1-score) 27

List of Table

Table 1.Cluster Purity-Based Region Assignment and Pseudo-Labeling Strategy 15
Table 2. Types of malicious traffic in UNSW-NB15 and CICIDS2017 datasets. 20
Table 3. Performance comparison on UNSW-NB15 (1% labeled) 24
Table 4 . Performance comparison of ablation study on UNSW-NB15 (1% labeled) 25
Table 5. Performance comparison of SMOTE variants on UNSW-NB15 (1% labeled)..... 27
Table 6. Performance comparison on CICIDS2017 (1% labeled)..... 28

Chapter1: Introduction

With the continuous evolution of networking technologies and the ongoing trend of global digitalization, the Internet now plays an integral role in almost every aspect of modern life, from personal communications and e-commerce to the monitoring and control of critical infrastructure. However, the Internet's openness and rapid expansion have brought about unprecedented cybersecurity challenges, especially as data volumes and the number of connected devices surge [1]. In recent years, threats such as hacking, malware, ransomware, and penetration attacks have emerged one after another, becoming increasingly stealthy, diverse, and automated through techniques like polymorphism, automated attack toolkits, and Ransomware-as-a-Service (RaaS). These developments pose significant obstacles to traditional signature-based intrusion detection systems (IDS), which often fail to detect unknown or zero-day attacks in a timely manner [2, 3]. To address these limitations, both academia and industry are exploring adaptive solutions—such as behavioral analysis, anomaly detection, and machine/deep learning approaches—to complement or even replace traditional methods, thereby improving detection accuracy and robustness [4-6].

Supervised learning models have demonstrated strong classification performance on standard intrusion detection datasets [7–9]. However, they generally require large amounts of balanced and representative labeled data for effective training [10]. In real-world scenarios, such data are difficult to obtain due to heterogeneous traffic sources, the high cost of manual annotation, and the labor-intensive process of validating each flow. Moreover, since malicious traffic naturally occurs far less frequently than benign traffic, the limited labeled datasets are often highly imbalanced. This class imbalance causes models to overfit the majority (normal) class, suppressing their ability to recognize minority-class attacks and degrading overall detection performance.

Semi-supervised learning has therefore attracted considerable interest as an effective approach to addressing both label scarcity and limited resources. Broad categories of semi-supervised methods include consistency regularization, co-training, and graph-based approaches, each of which leverages unlabeled data differently. Among these, self-training has become one of the most widely adopted strategies due to its conceptual simplicity and ease of integration. It begins by training an initial model on a small labeled dataset, then uses

the model to assign pseudo-labels to unlabeled samples with high confidence, iteratively incorporating these samples into the training set to refine the model's performance [11]. This method is conceptually simple, integrates seamlessly with existing architectures, and can leverage large amounts of unlabeled data. However, when applied to IDS, self-training still faces a critical challenge: the severe class imbalance in the unlabeled data. Because abnormal (attack) traffic typically constitutes only a small fraction of network flows, the pseudo-labels generated are often biased toward the majority (normal) class. This bias causes the model to focus primarily on normal samples, reducing its effectiveness in detecting rare or subtle attacks [12, 13].

To overcome these issues, we propose a Regional Self-Training framework that integrates density-aware clustering with targeted data augmentation to produce higher-quality pseudo-labels and enhance the representation of minority-class samples. First, HDBSCAN [14] is used to partition the combined labeled and unlabeled data into coherent density-based regions, and pseudo-labels are generated only in high-confidence clusters to reduce the risk of mislabeling. Second, for minority-class regions with sparse samples and scattered distribution, the SMOTE [15] technique is applied for synthetic oversampling to improve their representativeness in the training set, thereby mitigating the adverse effects of class imbalance on model learning. By jointly considering the intrinsic organization of the data and the distribution patterns across categories, our method enables the model to better capture the distinctive characteristics of minority-class attacks, ultimately improving overall detection capability.

To evaluate the proposed framework, we conducted experiments on the UNSW-NB15 and CICIDS2017 datasets, simulating attack behavior and capturing traffic patterns in realistic network environments. In these experiments, only 1% of the data was labeled to reflect the scarcity of labeling resources, while the original benign-to-malicious ratios—approximately 87:13 in UNSW-NB15 and 80:20 in CICIDS2017—were preserved to maintain the inherent class imbalance. This setting allows us to assess the framework's effectiveness under both limited labeled data and skewed class distributions. Experimental results show that our method surpasses traditional self-training approaches in overall classification performance and demonstrates strong generalization ability with promising applicability to real-world intrusion detection scenarios.

Chapter2: Background and Related Works

2.1 Background

This section introduces the key techniques that form the foundation of our proposed method. We begin with an overview of self-training, a widely used semi-supervised learning strategy that leverages unlabeled data by iteratively refining the model with pseudo-labeled examples. Next, we explain the concept of pseudo labeling, which plays a central role in self-training frameworks. To improve the quality of pseudo-labeling under class-imbalanced scenarios, we explore HDBSCAN, a density-based clustering algorithm capable of identifying structurally coherent clusters in unlabeled data. Finally, we introduce SMOTE, an oversampling technique designed to balance minority and majority classes by generating synthetic examples for underrepresented categories. Together, these techniques enable robust learning from limited labeled data and guide the design of our method.

2.1.1 Self-Training

Self-training is a type of semi-supervised learning. The earliest concept was first proposed by David Yarowsky [16], and many subsequent studies have implemented self-training based on his ideas. Building on this foundation, numerous scholars have explored and analyzed the mechanisms and effects of self-training in depth [17], pointing out that this method predicts unlabeled data using the model itself, generates pseudo-labels to expand the training dataset, and thereby improves learning efficiency.

The self-training process, illustrated in Figure 1, can be broadly divided into four steps: A, initial training, where a small portion of labeled data is used to train a base model; B, pseudo-label prediction, where the trained model is applied to unlabeled data to generate predicted labels; C, high-confidence sample selection, where only samples with sufficiently high prediction confidence are retained as pseudo-labeled data; and D, iterative retraining, where the pseudo-labeled data are combined with the original labeled dataset and the model is retrained. Steps B to D are repeated until the model converges or reaches the preset termination condition.

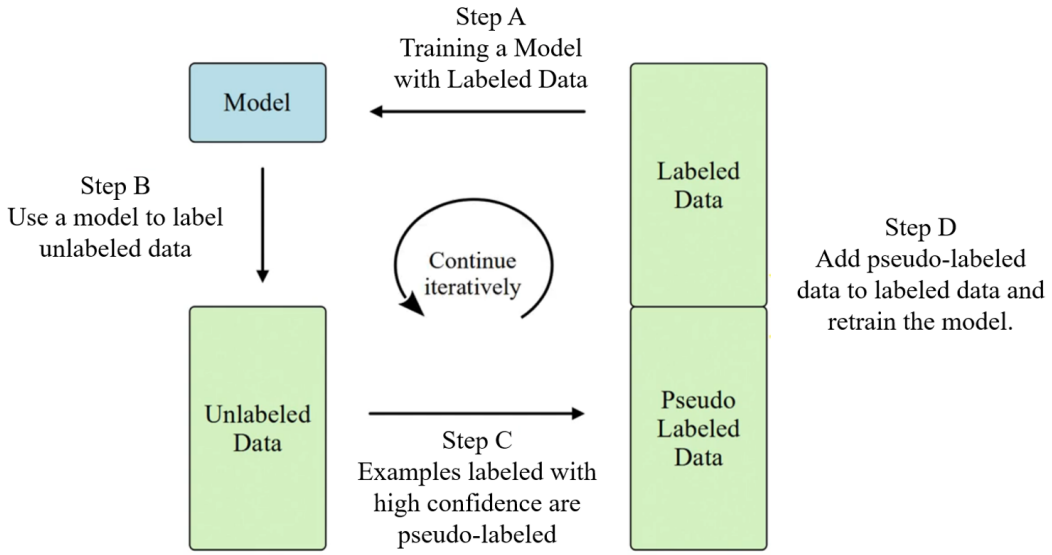

Figure 1 Flowchart of the Self-Training Process

Based on the above process, self-training offers several advantages, especially in the context of intrusion detection systems (IDS) [18]. It reduces the reliance on large amounts of manually labeled data by allowing the model to learn from unlabeled samples through its own predictions, which makes it attractive in low-supervision environments. However, when applied to IDS, this benefit is often limited by the severe class imbalance in network traffic, since abnormal events in real-world environments are rare, hard to label, and costly. Despite this limitation, self-training remains appealing because it can continuously expand the training dataset through iteration, enabling the model to gradually improve its understanding of new samples. This iterative process helps capture potential attack patterns and enhances the detection of unknown or evolving threats.

2.1.2 Pseudo Labeling

Pseudo labeling [19] is a crucial component of self-training, as it determines whether unlabeled data can be effectively incorporated into the learning process. The core idea is to treat the model's high-confidence predictions on unlabeled samples as "pseudo labels" and reuse these samples in subsequent training rounds. In a typical classification task, the output of the final neural network layer is passed through the Softmax function [20] to produce a probability distribution over classes, representing the model's confidence in each category. The class with the highest probability is assigned as the predicted label, and the corresponding probability—derived from the Softmax output—is regarded as the

confidence score. This mechanism allows pseudo-labeling to select predictions with high confidence, assigns them pseudo labels, and includes the corresponding samples in the next round of training.

$$\mathrm{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}}$$

However, since these pseudo labels are generated by the model itself rather than manually annotated, they are easily influenced by the initial model bias or class imbalance, causing pseudo-labels to be biased toward the majority class, thereby amplifying existing distributional skew. Such bias not only reduces the overall learning effectiveness but can also cause error accumulation through repeated training iterations. Therefore, improving the accuracy and representativeness of pseudo labels has become a critical challenge in self-training approaches [21].

2.1.3 HDBSCAN

HDBSCAN [14] (Hierarchical Density-Based Spatial Clustering of Applications with Noise) is a density-based hierarchical clustering method that extends and improves upon the classic DBSCAN [22] algorithm. Compared with DBSCAN, HDBSCAN offers a more automated clustering process, does not require the number of clusters to be specified in advance, and is better suited for datasets with complex structures, high diversity, or irregular shapes.

The core concept of HDBSCAN is the combination of density estimation and hierarchical clustering. During clustering, the algorithm adjusts the relative distance between data points based on the density of each point's neighborhood: if a point is located in a low-density region, its distance to neighboring points is increased; conversely, in high-density regions, the distance remains relatively stable, as illustrated in Figure 2. This mechanism helps accurately reveal the underlying structure in the data and avoids misclassifying sparse regions as valid clusters.

HDBSCAN is a more rigorous clustering technique that can automatically group data with similar characteristics and determine an appropriate number of clusters based on the dataset's inherent density structure. One of HDBSCAN's key advantages is its ability to handle clusters with arbitrary shapes, imposing minimal constraints on the data's shape and

distribution. As a result, it is more flexible than traditional methods such as K-means or DBSCAN and is applicable to a broader range of data types and practical scenarios.

Overall, HDBSCAN is particularly effective for datasets with complex or heterogeneous structures, and it has gradually become an indispensable tool in modern clustering analysis. Figure 3 presents a performance comparison between HDBSCAN and other mainstream clustering algorithms, where HDBSCAN demonstrates strong results across multiple evaluation metrics, highlighting its potential and robustness in practical applications.

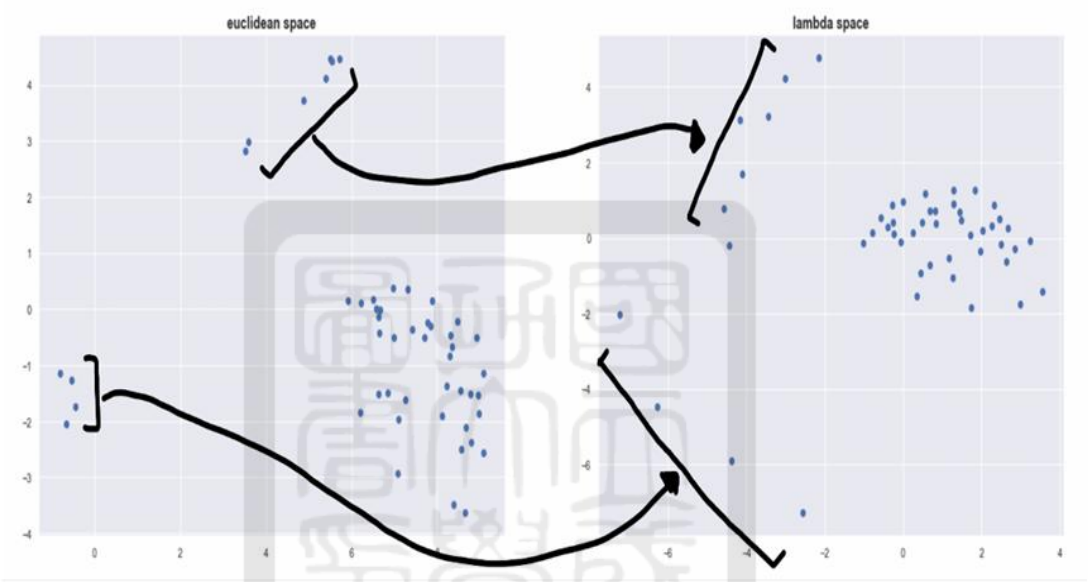

Figure 2. Euclidean space vs. λ-space in HDBSCAN [23]

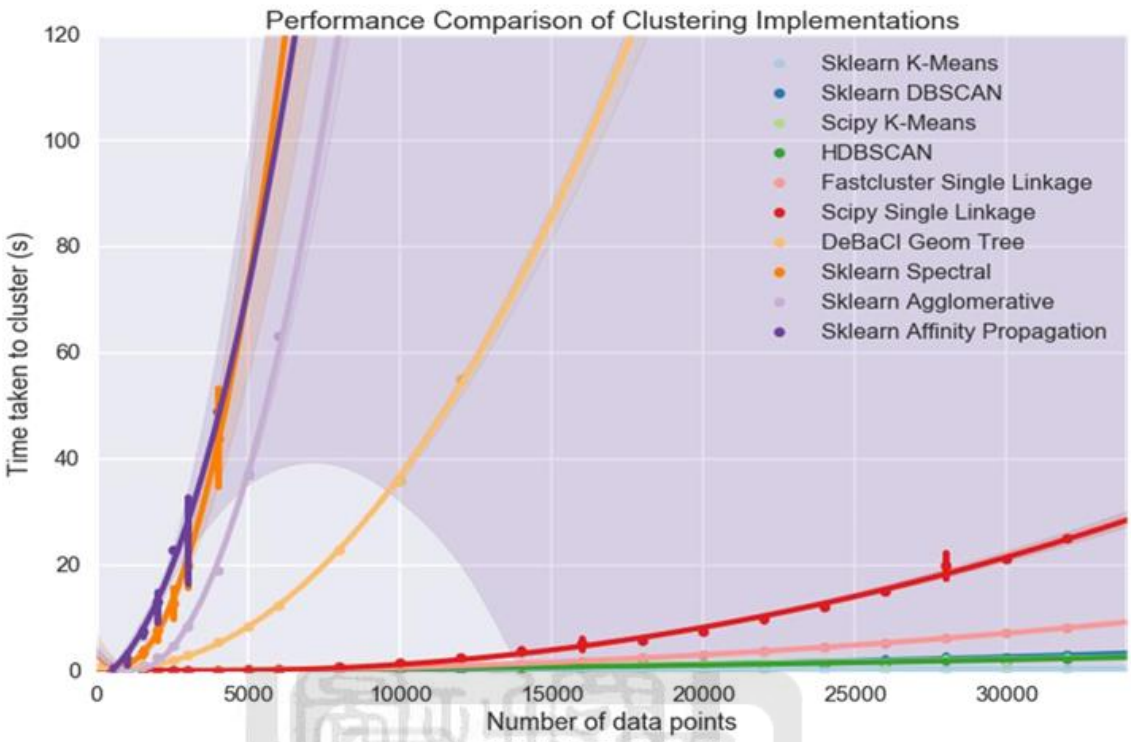

Figure 3. Performance of clustering algorithms [24]

2.1.4 SMOTE

SMOTE [15] (Synthetic Minority Over-sampling Technique) is a method designed to address the issue of class imbalance, originally introduced by Chawla et al. in 2002. Rather than duplicating existing instances, the key idea behind SMOTE is to expand the representation of the underrepresented class by producing artificially generated data points. This increases the training diversity and enhances the classifier's performance when handling imbalanced datasets.

Unlike conventional oversampling methods that simply replicate existing data, SMOTE generates artificial instances by interpolating between a sample from the minority group and its k-nearest neighbors in the feature space. Specifically, for each such sample, the algorithm chooses one of its neighbors and synthesizes a new data point positioned randomly along the straight line between them. By enriching the variability within the minor class, this strategy maintains the overall distribution while improving the model's ability to generalize. This is illustrated in Figure 4.

The primary benefit of SMOTE lies in its capacity to reduce the effects of class imbalance and improve the recognition of rare categories. However, SMOTE may generate unreliable samples, particularly near decision boundaries or when the dataset contains outliers, which may lead to the generation of low-quality synthetic samples. To address these issues, SMOTE is often used in combination with other techniques—such as data cleaning algorithms or ensemble classifiers—to improve overall performance and robustness. Overall, SMOTE is regarded as a classic and widely adopted solution for handling class imbalance. It has been successfully applied in various fields, including intrusion detection, medical diagnosis, and fraud detection, and remains a vital tool in modern data preprocessing pipelines.

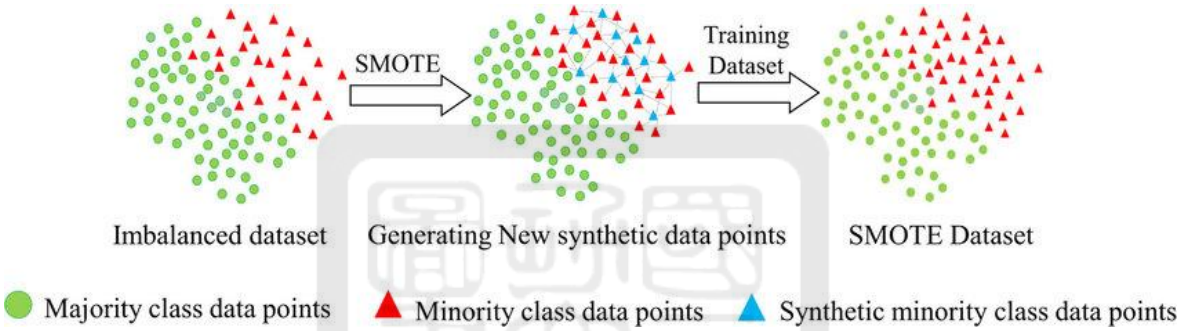

Figure 4. SMOTE Process [15]

2.2 Related Works

This section surveys recent applications of machine learning and semi-supervised learning for intrusion detection systems (IDS), emphasizing the challenges posed by class imbalance in unlabeled data within self-training approaches.

2.2.1 Limitations of Traditional IDS

As network attacks become increasingly diverse and covert, traditional IDS that rely on rules or matching signature mechanisms have difficulty effectively identifying unknown or variant attacks [25]. To overcome this limitation, more and more studies have tried to apply machine learning technology to IDS architecture, hoping to automatically learn attack behavior characteristics and classify them through data-driven methods [26]. Among them, supervised learning methods, such as support vector machines (SVM), random forests, and deep neural networks (DNN), have shown excellent classification results in multiple standard datasets (such as KDD99, NSL-KDD, CICIDS2017). However, such models often rely heavily on large amounts of well-distributed and representative labeled data during training [27]. In actual scenarios, due to the extremely low proportion of abnormal samples and the high cost of manual labeling, acquiring labeled data is challenging, which consequently leads the model to be biased toward the majority class（normal） [26,28]. The recognition effect is significantly reduced.

2.2.2 Semi-Supervised Learning in IDS

To mitigate the limitation of scarce labeled resources, semi-supervised learning has become an active research direction in IDS [11,26]. Wagh and Jabbar (2014) [29] applied a self-training framework on the NSL-KDD dataset and achieved an improvement of approximately 8% in detection rate using only 5% of labeled data, by repeatedly incorporating high-confidence pseudo-labeled samples into the training set. However, this method generally assumes that labeled and unlabeled data share similar distributions and does not address the potential class imbalance problem.

Liu et al. (2024) proposed a semi-supervised encrypted traffic detection method [30] that fuses sequential and graph-structural features via co-training, using autoencoder reconstruction error as a regularize and focal loss for hard-sample emphasis. On UNSW-NB15 and CICIDS2017 with just 1% labeled data, it achieves a 3.49% F1-score

improvement over baselines, demonstrating robust performance in imbalanced, encrypted traffic scenarios.

Within the spectrum of semi-supervised methods, self-training stands out as one of the most widely adopted approaches in IDS, valued for its ability to iteratively expand training data and adapt to evolving attack patterns. However, its reliance on pseudo-labels also exposes it to the challenges of class imbalance, making it a central focus of recent studies.

2.2.3 Self-training under Class Imbalance

Ashfaq et al. (2017) [31] pointed out that if all unlabeled data is directly included in the training, many normal samples will be mislabeled, further weakening the model's ability to identify minority-class attacks. To this end, they proposed using fuzziness as a classification confidence measurement indicator and only selected low-fuzziness or high-confidence samples for pseudo-labeling to reduce labeling errors. Experiments on the NSL-KDD dataset show that compared with the unscreened self-training strategy, this method improves the overall detection rate by about 5%, and the F1-score of minority-class attack categories by about 7% on average.

Zheng et al. proposed the SF-IDS framework [12], aiming to address pseudo-labeling errors and severe class imbalance in intrusion detection tasks. The study highlights that in standard self-training workflows, an overwhelming proportion of majority-class samples in unlabeled data can cause pseudo-labels to be overly biased toward frequent classes, thus degrading the model’s ability to detect minority-class attacks. To mitigate this issue, SF-IDS employs a dual pseudo-label screening strategy that integrates prediction probability and uncertainty estimation to enhance label reliability. Moreover, the framework introduces customized loss functions to enhance minority-class learning and applies Borderline-SMOTE to oversample pseudo-labeled minority samples. To further address data imbalance, the authors apply Borderline-SMOTE to oversample the pseudo-labeled minority-class samples. Experiments conducted on the NSL-KDD and CICIDS2017 datasets show that, with only 1% labeled data, SF-IDS achieved Macro-F1 improvements of 3.01% and 2.71%, respectively. The recall of minority classes also improved significantly, demonstrating the framework’s effectiveness under highly imbalanced conditions.

Although methods like Borderline-SMOTE can mitigate class imbalance, they often ignore structural and density patterns in unlabeled data, generating unrealistic samples. Our framework addresses this by using HDBSCAN to find semantically coherent regions and applying localized SMOTE, preserving data structure and improving minority-class detection. Compared with SF-IDS, it emphasizes semantic cohesion and density-aware processing for a more robust IDS strategy.

Chapter3: Methodology

3.1 System Overview

The Regional Self-Training framework (Figure 5) begins by training an initial deep neural network (DNN) base model on a small labeled dataset. It then enters a self-training loop in which density-aware pseudo-labeling is applied: the combined labeled and unlabeled data are clustered using HDBSCAN, and pseudo-labels are assigned within both high- and medium-purity regions through cluster-based and model-based strategies. The generated pseudo-labels are further validated using a KNN consistency check to ensure reliability. Next, regional oversampling generates synthetic minority-class samples exclusively within those clusters to rebalance the training set. The model is then retrained on the expanded dataset, and this cycle of pseudo-labeling, verification, oversampling, and retraining repeats until convergence. Finally, the resulting model is evaluated on held-out test data.

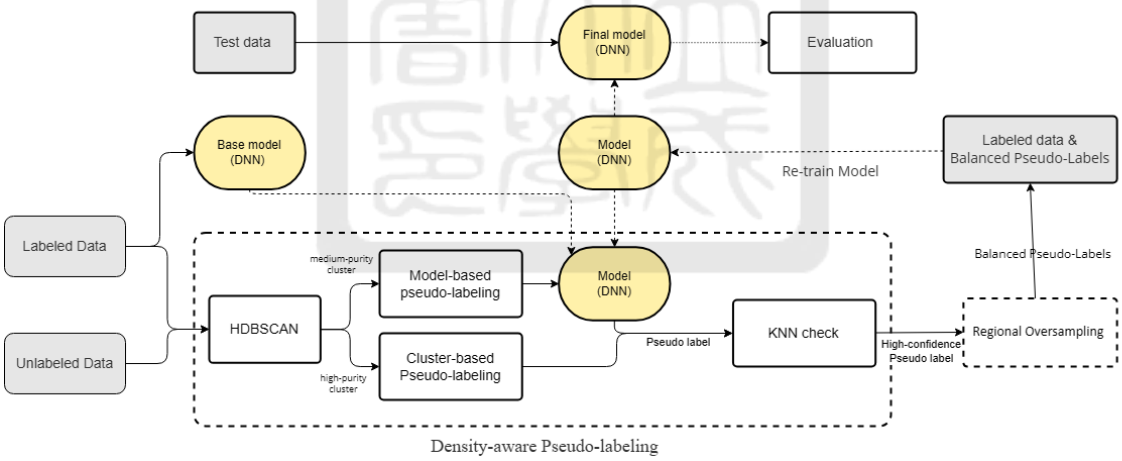

Figure 5. Regional Self-Training Framework System Overview

The following sections detail each component: Section 3.2 details Density-aware Pseudo-labeling, where HDBSCAN first clusters the data to estimate cluster purity, and pseudo-labels are then assigned via cluster-based or model-based strategies[羅大郡1][許2], further validated by a KNN check. Section 3.3 introduces Regional Oversampling to rebalance classes within clusters, while Section 3.4 summarizes the complete Regional

Self-Training algorithm.

3.2 Density-aware Pseudo-labeling

3.2.1 Clustering by HDBSCAN

We begin by feeding both labeled and unlabeled samples into HDBSCAN. Unlike partitioning-based clustering, HDBSCAN dynamically adjusts cluster boundaries according to local density variations, allowing it to group similar malicious flows together while maintaining clear separation between distinct attack types. This property makes it particularly suitable for network traffic, where malicious behaviors are diverse while benign traffic is generally more dispersed. As a result, heterogeneous malicious flows tend to form compact clusters, whereas benign flows coalesce into larger groups. Samples that lack sufficient density support are automatically designated as noise.

As illustrated in Figure 6, the left plot shows the original feature distribution—green for benign flows, yellow for malicious flows, and gray for unlabeled samples—while the right plot presents the clusters discovered by HDBSCAN.

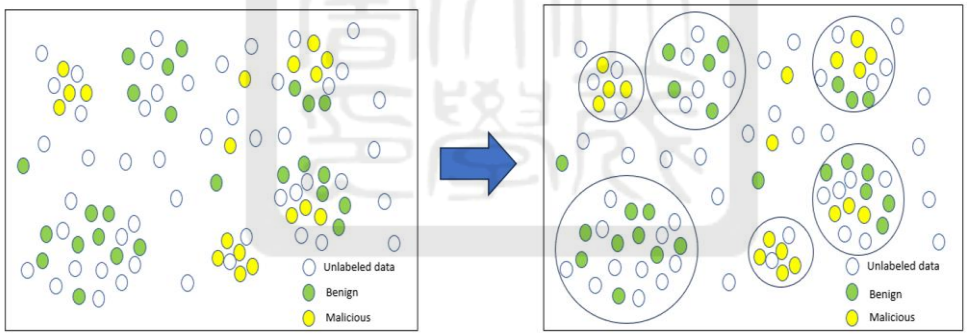

Figure 6.HDBSCAN Clustering Results

After clustering, each cluster is categorized into one of four cases based on its labeled-sample composition (Figure 7):

Case A: roughly equal numbers of benign and malicious labels,

Case B: one class clearly dominates but a minority remains,

Case C: almost entirely a single class, and

Case D: no labeled samples at all.

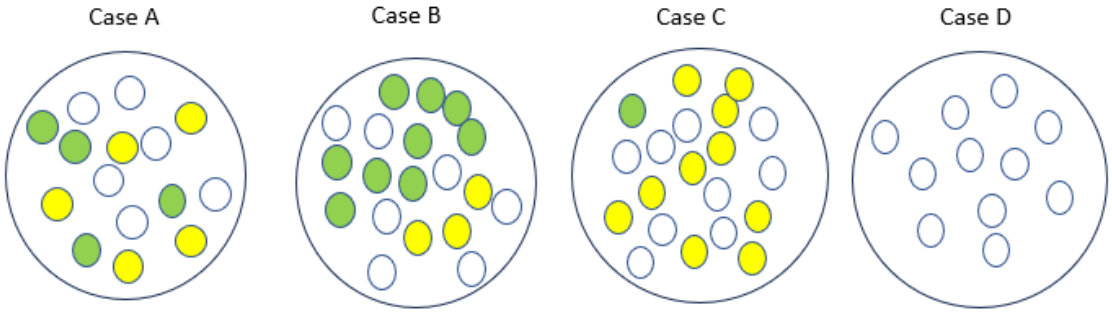

Figure 7. HDBSCAN Cluster Categories by Label Composition

3.2.2 Pseudo-label Assignment

Next, we compute each cluster's purity by dividing the number of labeled samples in the dominant class by the total number of labeled samples in that cluster (e.g., 0.55 for Case A, 0.75 for Case B, 0.90 for Case C). Based on these purity scores, clusters are assigned to three actionable regions—low, medium, or high purity (with "unknown" for Case D)—and distinct pseudo-labeling strategies are applied:

1. **High-Purity Region (purity ≥ 0.9):** all unlabeled samples inherit the cluster's majority label via majority vote (e.g., label all points "malicious").
2. **Medium-Purity Region (0.7 ≤ purity < 0.9):** we defer to the base model's predictions, assigning pseudo-labels according to its SoftMax outputs to capture boundary information without overcommitting to the majority.
3. **Low-Purity and Unknown Region (purity < 0.7 or no labels):** clusters are too mixed or unlabeled to trust, so we discard their unlabeled samples entirely.

$$\mathrm{Purity}_L(C) = \frac{\max_{c \in \mathcal{Y}} \left| \{ x_i \in L \mid y_i = c \} \right|}{|L|}$$

Here, L denotes the subset of labeled samples within a cluster, $|L|$ is the number of such labeled samples, $\mathcal{Y}$ is the set of all possible labels, y_i is the ground-truth label of sample x_i, and C represents the majority class assigned to cluster L.

Different purity regions require tailored strategies to balance accuracy and coverage. High-purity clusters, dominated by a single class, allow reliable majority-vote labeling with minimal noise. Medium-purity clusters still contain mixed labels, so pseudo-labels

rely on the model to preserve boundary information. Low-purity or unlabeled clusters lack sufficient class evidence, and their samples are discarded to avoid errors.

Table 1 provides a summary of the relationship between cluster cases, their purity ranges, region types, and the corresponding pseudo-labeling strategies.

Case	purity	Region Type	Pseudo-Labeling Strategy
A	0.55	Low-Purity Region	Discard all unlabeled samples
B	0.75	Medium-Purity Region	Assign labels via model SoftMax
C	0.9	High-Purity Region	Majority-vote label assignment

Table 1.Cluster Purity-Based Region Assignment and Pseudo-Labeling Strategy

3.2.3 Pseudo-label Validation via KNN check

In the previous step, unlabeled samples were clustered using HDBSCAN and assigned pseudo-labels according to region-specific strategies: high-purity clusters by majority vote, medium-purity clusters by model predictions, while low-purity or unlabeled clusters were discarded.

Although these strategies mitigate labeling noise, incorrect pseudo-labels may still remain within clusters. To further enhance reliability, we apply a k-nearest neighbors (KNN) check. For each pseudo-labeled sample, its five nearest neighbors are checked; if more than four neighbors have inconsistent labels, the sample is deemed unreliable and removed.

As shown in Figure 8, the left plot presents pseudo-label assignment before validation, while the right plot shows the results after applying the KNN check, where noisy pseudo-labels have been removed.

Figure 8. Pseudo-label refinement through KNN check[羅大郡3][許4]

3.3 Regional Oversampling

After assigning reliable pseudo-labels to unlabeled samples in high- and medium-purity clusters, we rebalanced the training set using regional oversampling. Unlike global SMOTE, which generates minority-class samples across the entire pseudo-labeled set, our approach limits SMOTE to individual clusters, preserving local data structures and preventing the creation of unrealistic samples that span decision boundaries.

In the Medium-Purity Region, where clusters exhibit a clear majority class alongside a significant minority, we apply in-cluster SMOTE exclusively to the minority samples. This targeted oversampling synthesizes additional minority-class points near the decision boundary, balancing class proportions within each cluster and enhancing the model's ability to detect underrepresented attack flows (see Figure 9).

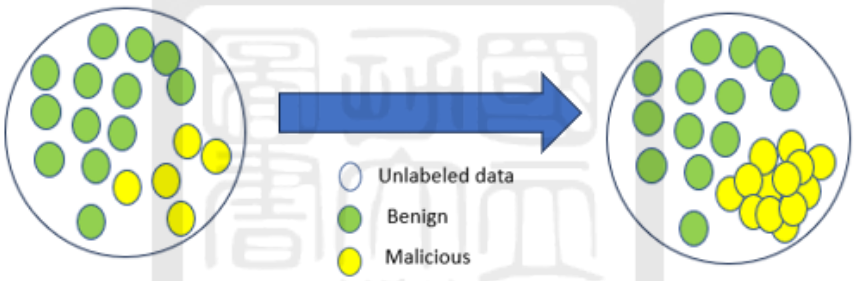

Figure 9.In-Cluster SMOTE for Minority Class Augmentation in Medium-Purity Regions

In the High-Purity Region, where each cluster is dominated by a single class, we correct for imbalance by oversampling minority clusters in proportion to the global pseudo-label distribution. We first aggregate pseudo-labeled counts from all high-purity clusters to compute the overall majority-to-minority ratio. For example, if four clusters together contain 30 benign and 10 malicious samples (a 3:1 ratio), we apply in-cluster SMOTE to each malicious cluster to triple the malicious instances, aligning their count with that of the majority class. This targeted, region-specific augmentation enhances minority-class representation without inundating the training set with majority pseudo-labels during retraining (see Figure 10).

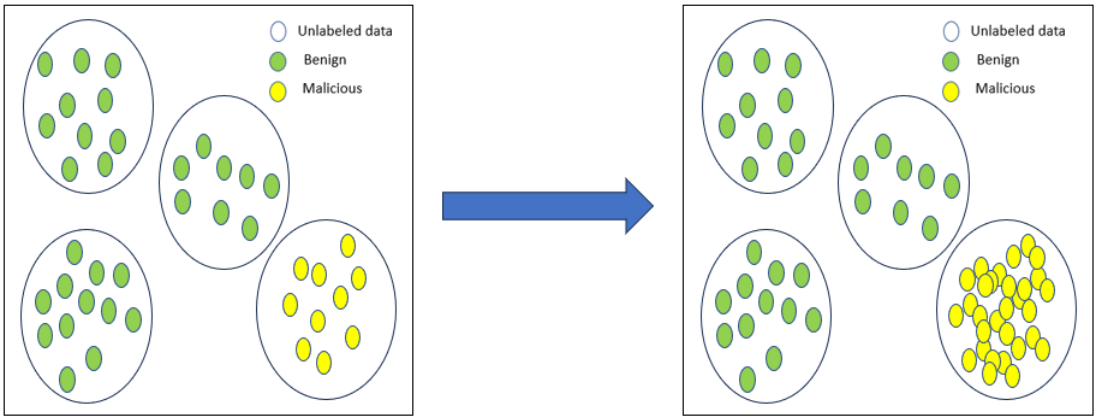

Figure 10. Proportional In-Cluster SMOTE in High-Purity Minority Clusters

After completing density-aware pseudo-labeling and regional oversampling, we merged the original labeled data, high-confidence pseudo-labels, and newly generated synthetic samples to form an expanded training set for retraining the base model. The entire self-training cycle—clustering, pseudo-labeling, oversampling, and retraining—repeats until validation performance stabilizes or meets a predefined convergence criterion.

3.4 Algorithm

To summarize the proposed method, Figure 11 presents the pseudocode of the complete Regional Self-Training workflow. The algorithm starts with supervised training on a small labeled dataset, then iteratively performs HDBSCAN clustering, purity assessment, pseudo-label assignment using region-based strategies, confidence filtering, and a KNN check to validate pseudo-label correctness, followed by in-cluster SMOTE oversampling and model retraining. This cycle repeats until convergence, after which the trained model is evaluated on the test set.

Algorithm 1: Regional Self-Training Framework

Input: Labeled data $D_l = \{(x_i, y_i)\}_{i=1}^{n}$, Unlabeled data $D_u = \{x_j\}_{j=n+1}^{n+m}$, Initial model M_0, Purity thresholds τ_{med}, τ_{high}, Confidence threshold θ, KNN neighbor count k_{nn}

Output: Trained model M^* and augmented labeled set D_l^*

```
M ← Train(M_0, D_l)
while stopping criterion not met do
    C ← HDBSCAN(D_l ∪ D_u) foreach cluster c ∈ C do
        p(c) ← max_y{(x, y) ∈ c ∩ D_l}c ∩ D_l if p(c) ≥ τ_high then
            add c to HighPurityRegions
        else
            if p(c) ≥ τ_med then
                add c to MedPurityRegions
            else
                discard cluster c
    foreach cluster c ∈ HighPurityRegions do
        ∀x ∈ c ∩ D_u : y*(x) ← majorityLabel(c ∩ D_l)
    foreach cluster c ∈ MedPurityRegions do
        ∀x ∈ c ∩ D_u : y*(x) ← arg max M(x)
    D_ps ← {(x, y*(x)) : x ∈ D_u, M(x)[y*(x)] ≥ θ}
    D_ps ← KNNCheck(D_ps, D_l, k_nn)
    D_syn ← InClusterSMOTE(D_ps, C)
    D_l ← D_l ∪ D_ps ∪ D_syn M ← Train(M, D_l)
M* ← M return M*, D_l
```

Figure 11. Algorithm for Regional self-training Framework

Chapter 4: Experiment

In this chapter, we validate the proposed Regional Self-Training method using two public datasets. We first conduct comprehensive experiments on UNSW-NB15 [32], including overall performance comparison with traditional self-training and other benchmark models using only 1% labeled data, ablation studies to evaluate the contribution of each component, and comparisons between our in-cluster SMOTE and conventional SMOTE. Finally, we verify the effectiveness and adaptability of our method on the CICIDS2017 [33] dataset, which contains highly imbalanced unlabeled data, and benchmark it against existing semi-supervised learning techniques.

4.1 Experiment Setup

In this section, we first describe the datasets and preprocessing steps, followed by the hardware and software environment, the base model structure, and the evaluation criteria.

4.1.1 Datasets

In this study, we selected two publicly available datasets that are widely used in intrusion detection research. To focus on intrusion detection rather than attack-type classification, all attack categories were merged into a single malicious class, resulting in a binary classification setting (benign vs. malicious). Each dataset was then split into 1% labeled, 79% unlabeled, and 20% test subsets.

UNSW-NB15[32] was developed by the University of New South Wales in collaboration with the Defence Science and Technology Organisation (DSTO) in 2015. It contains 2,540,044 records covering nine categories of attacks, including Fuzzers, Analysis, Backdoor, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms, and normal traffic. Normal traffic makes up about 87.4% of the dataset, while malicious traffic represents 12.6% (see Figure 12). The malicious traffic categories are also detailed in Table 2.

CICIDS2017[33] was collected by the Canadian Institute for Cybersecurity (CIC) and the University of New Brunswick in a campus network environment from July to August 2017, simulating various attacks such as DoS, brute-force, web attacks, and botnet. The dataset comprises approximately 2,830,000 records, of which normal traffic accounts for

about 80.2% and malicious traffic for about 19.8% (see Figure 12). The malicious traffic categories—DoS, Brute Force, Web Attacks, Botnet, Infiltration, SSH Brute Force, Heartbleed, Port Scan, and GoldenEye—are detailed in Table 2.

The markedly different class–imbalance ratios and attack scenarios in these two datasets allow us to thoroughly assess how robust and generalizable the proposed method is under different imbalance conditions.

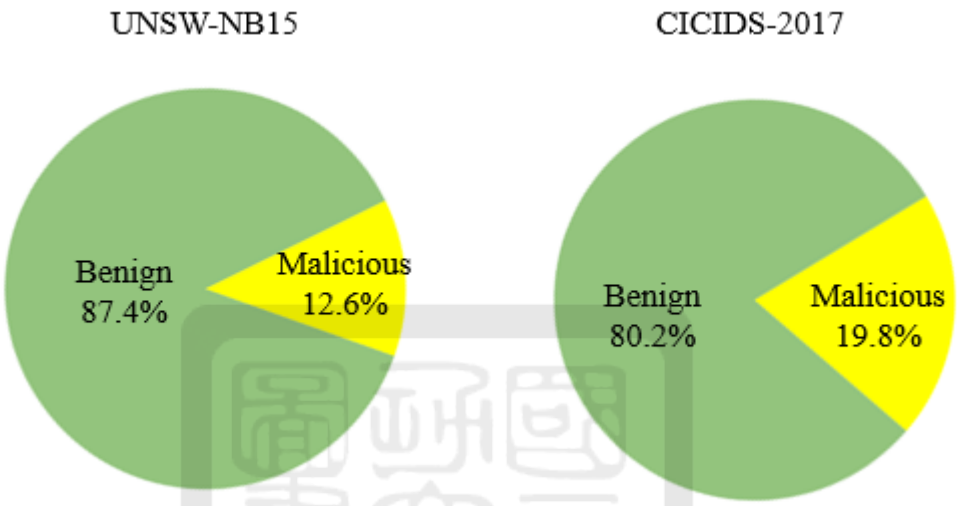

Figure 12. Normal vs. Malicious Traffic Proportions in UNSW-NB15 and CICIDS2017

UNSW-NB15 Malicious Type	**CICIDS2017 Malicious Type**
Generic	DoS Hulk
Exploits	PortScan
DoS	DDoS
Fuzzers	DoS GoldenEye
Reconnaissance	FTP-Patator
Analysis	SSH-Patator
Backdoors	DoS slowloris
Shellcode	DoS Slowhttptest
Worms	Bot
	Web Attack – Brute Force
	Web Attack – XSS
	Web Attack – SQL Injection
	Infiltration
	Heartbleed

Table 2. Types of malicious traffic in UNSW-NB15 and CICIDS2017 datasets.

4.1.2 Data Preprocessing

For both UNSW-NB15 and CICIDS2017, we applied an identical preprocessing pipeline to ensure consistency. First, we removed any records containing missing, NaN, or infinite values (affecting less than 0.1% of the samples). Features that could reveal distributional or temporal information—such as IP addresses and timestamps—were also discarded. Next, all continuous features (e.g., packet length, flow duration, byte count) were standardized to zero mean and unit variance. To focus on intrusion detection rather than attack type classification, all attack categories were merged into a single malicious class, resulting in a binary classification setting (benign vs. malicious). Finally, we constructed the 1% / 79% / 20% split by randomly sampling the processed data, ensuring that the labeled, unlabeled, and test subsets were mutually exclusive and preserved the original class-imbalance characteristics.

4.1.3 Environment

Our hardware setup includes an NVIDIA GeForce RTX 2080 Ti GPU and an Intel® Core™ i7-7700 CPU running at 3.6 GHz. All experiments were conducted in Jupyter Notebook using Python 3.10.20 and TensorFlow 2.10.0.

4.1.4 Base Model Structure

Our base model is a fully connected feed-forward network with six successive hidden layers of 256 units each, using ReLU activations and He-normal weight initialization. To reduce overfitting, dropout with a rate of 0.4 is applied after the fourth, fifth, and sixth hidden layers. A final dense layer with two output units produces the logits for the "Benign" and "Malicious" classes. This balanced design—combining ample representational capacity with regularization—serves as the backbone for both initial supervised training and subsequent self-training iterations (see Figure 13).

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 43)]	0
dense (Dense)	(None, 256)	11264
dense_1 (Dense)	(None, 256)	65792
dense_2 (Dense)	(None, 256)	65792
dense_3 (Dense)	(None, 256)	65792
dropout (Dropout)	(None, 256)	0
dense_4 (Dense)	(None, 256)	65792
dropout_1 (Dropout)	(None, 256)	0
dense_5 (Dense)	(None, 256)	65792
dropout_2 (Dropout)	(None, 256)	0
dense_6 (Dense)	(None, 2)	514

Figure 13. Base Model Architecture

4.1.5 Evaluation Criteria

In this section, we describe the three primary metrics used for evaluation: precision, recall, and F1-score. Based on the model's classifications of benign and malicious traffic, predictions fall into four categories:

- **True Positives (TP)**: malicious traffic correctly identified.
- **False Positives (FP)**: benign traffic incorrectly labeled as malicious.
- **True Negatives (TN)**: benign traffic correctly identified.
- **False Negatives (FN)**: malicious traffic missed.

Precision is defined as:

$$Precision = \frac{TP}{TP + FP}$$

It quantifies the proportion of correctly detected malicious flows among all flows predicted as malicious.

Recall is defined as:

$$Recall = \frac{TP}{TP + FN}$$

It captures the proportion of actual malicious flows that are correctly detected.

Because both false alarms and missed detections carry significant risks in malicious detection, we use the F1-score to provide a balanced measure that integrates precision and recall into a single metric:

$$F1\text{-}score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

4.2 Experiment Result

4.2.1 Experiment 1: Binary Classification on UNSW-NB15[5][6][7]

In this experiment, we use the standard self-training approach (denoted as Standard-ST) as our baseline and additionally compare against the recent SSET-MTC method [30] (2024). On the UNSW-NB15 dataset, using only 1% labeled data, our Regional Self-Training achieves the highest performance among all approaches. Standard-ST obtains 92.7% precision, 94.5% recall, and a 93.6% F1-score, while incorporating SSET-MTC raises these to 95.28%, 95.36%, and 95.32%, respectively. Our approach further improves performance to 95.6% precision, 98% recall, and a 96.8% F1-score, as shown in Table 3.

Compared with existing approaches, our method demonstrates clear advantages. Standard-ST typically relies on Softmax confidence for pseudo-labeling, which risks propagating noisy labels and amplifying majority-class bias. In contrast, our framework integrates density-aware pseudo-labeling with KNN check and regional oversampling, improving pseudo-label reliability and mitigating imbalance, all while keeping the base model unchanged. By comparison, SSET-MTC improves detection through multi-task learning and semantic consistency but requires substantial architectural redesign and additional training objectives. These results highlight the effectiveness of our method, delivering substantial performance gains without requiring architectural changes to the base model.

Method	Precision	Recall	F1-score	Accuracy
Standard-ST [19]	92.7%	94.5%	93.6%	98.4%
SSET-MTC [30]	95.3%	95.4%	95.3%	95.9%
Our proposed	95.6%	98%	96.8%	99.2%

Table 3. Performance comparison on UNSW-NB15 (1% labeled)

4.2.2 Experiment 2: Ablation Study on UNSW-NB15

To evaluate the contribution of each component in our Regional Self-Training framework, we conduct an ablation study on the UNSW-NB15 dataset using only 1% labeled data. Four configurations are compared:

- **Standard self-training (Standard-ST)** – the baseline self-training framework without our proposed modifications.
- **Density-Aware Pseudo-Labeling only (DAP only) –** replacing the baseline pseudo-labeling with our density-aware approach, while omitting regional oversampling.
- **Regional Oversampling only (RO only)**– applying regional in-cluster SMOTE on baseline pseudo-labels without density-aware filtering.
- **All components without KNN check (All w/o KNN) –** integrating both density-aware pseudo-labeling and regional oversampling while skipping the KNN check step to assess its effectiveness.
- **All components**– integrating both density-aware pseudo-labeling and regional oversampling, i.e., our completely proposed framework.

As shown in Figure 14, both DAP-only and RO-only individually improve the F1-score compared to the baseline, indicating that each component contributes independent performance gains. The "All w/o KNN" setting further shows that combining density-aware pseudo-labeling (without the KNN check) and regional oversampling already achieves substantial improvements (96.30%). When the KNN check is enabled within the density-aware pseudo-labeling process, the performance increases even further.

Table 4 provides a more detailed view of the results, showing that DAP-only mainly emphasizes Precision improvements (96.47%), whereas RO-only primarily boosts Recall

(96.69%). The complete framework (with both DAP including KNN check and RO) achieves the best balance across all metrics, with Precision 95.62%, Recall 98.04%, and F1-score 96.81%, confirming that density-aware pseudo-labeling (enhanced by KNN check) and regional oversampling are complementary in improving detection performance on imbalanced unlabeled data.

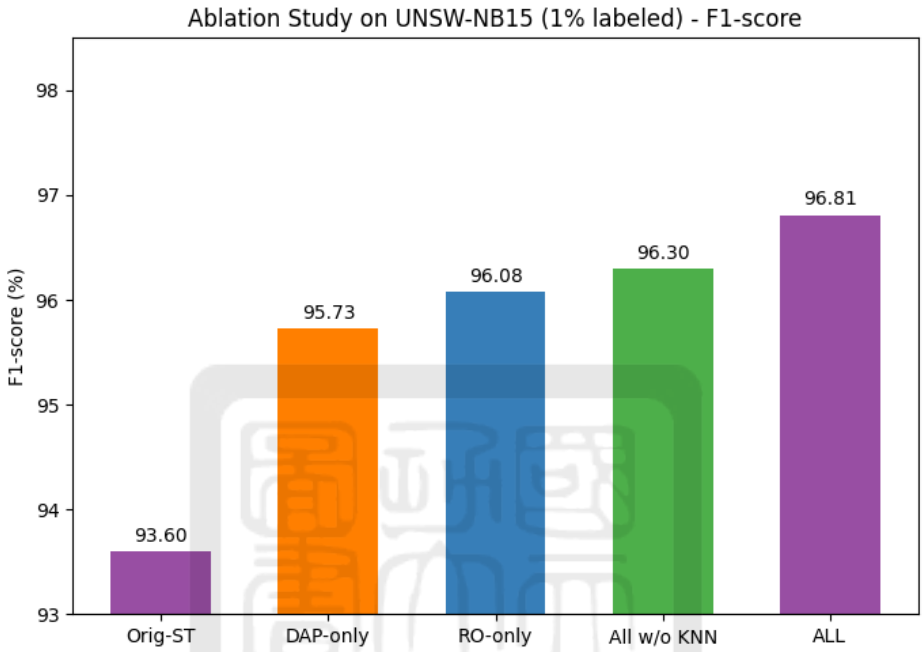

Figure 14. Ablation Study Results (F1-score)

Method	**Precision**	**Recall**	**F1-score**	**Accuracy**
Standard-ST[19]	92.7%	94.5%	93.6%	98.4%
DAP only	96.5%	95.0%	95.7%	98.9%
RO only	95.5%	96.7%	96.1%	99.0%
All w/o KNN	95.6 %	97.0%	96.3%	99.1%
All components	95.6%	98.0%	96.8%	99.2%

Table 4 . Performance comparison of ablation study on UNSW-NB15 (1% labeled)

4.2.3 Experiment 3: Comparison of Different SMOTE Variants

To evaluate the effectiveness of our proposed In-cluster SMOTE strategy, we compare it with three commonly used oversampling methods within the self-training framework on the UNSW-NB15 dataset using only 1% labeled data. The evaluated methods include:

- **No oversampling (Standard-ST** [19]**)** – the baseline self-training framework without any oversampling.
- **SMOTE** [15]– applies global SMOTE across the entire pseudo-labeled set.
- **Borderline-SMOTE** [34] –generates synthetic samples near the decision boundary.
- **ADASYN** [35] – adaptively generates more samples for harder-to-learn minority examples.
- **In-cluster SMOTE**– our proposed method, applying SMOTE within each cluster based on the regional purity and distribution.

As shown in Table 5 and Figure 15, although all oversampling methods outperform the baseline, traditional approaches exhibit clear limitations. SMOTE and Borderline-SMOTE achieve very high recall (99.4% and 99.08%, respectively), but their precision drops noticeably (92.22% and 92.71%) due to synthetic samples generated without considering local distribution. ADASYN improves precision (95.39%) by adaptively focusing on harder-to-learn cases, yet its recall (96.86%) is relatively lower, suggesting insufficient coverage of minority examples.

In contrast, our proposed In-cluster SMOTE achieves a better balance, with both high precision (95.62%) and recall (98.04%), resulting in the best overall F1-score (96.81%). This advantage arises from generating synthetic samples only within high-purity clusters, which avoids the noise and boundary distortion introduced by global SMOTE while preserving minority-class structure more effectively than ADASYN. These findings suggest that our method not only strengthens minority-class representation but also ensures robust overall detection performance.

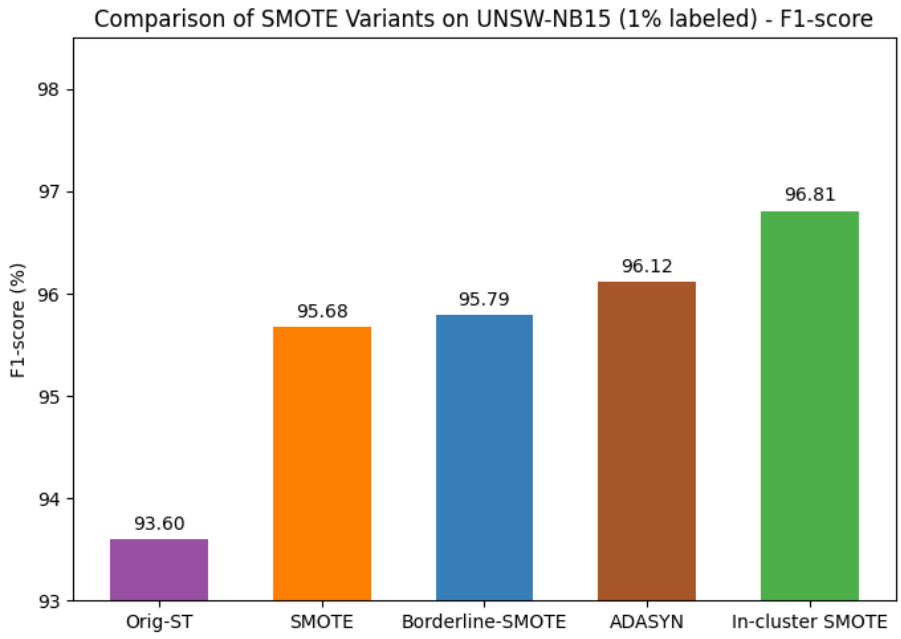

Figure 15. SMOTE Variant Comparison (F1-score)

Method	**Precision**	**Recall**	**F1-score**	**Accuracy**
Standard-ST [19]	92.7%	94.5%	93.6%	98.4%
SMOTE [15]	92.2%	99.4%	95.7%	98.9%
Borderline-SMOTE [34]	92.7%	99.1%	95.8%	98.9%
ADASYN [35]	95.4%	96.9%	96.1%	99.0%
In-cluster SMOTE	95.6%	98.0%	96.8%	99.2%

Table 5. Performance comparison of SMOTE variants on UNSW-NB15 (1% labeled)

4.2.4 Experiment 4: Binary Classification on CICIDS2017

On the CICIDS2017 dataset, Standard-ST, trained with only 1% labeled data, achieves 91.6% precision, 93.9% recall, and a 92.7% F1-score, but its performance is hindered by noisy pseudo-labels. Applying FixMatch improves the results to 93.9% precision, 93.6% recall, and a 93.8% F1-score by enforcing consistency, yet the gain remains limited as it does not explicitly address the severe class imbalance.

Compared with existing approaches, our method demonstrates clear advantages. Standard-ST relies on Softmax confidence for pseudo-labeling, which can propagate noisy labels, while FixMatch enforces consistency but does not explicitly address class imbalance. In contrast, our framework integrates density-aware pseudo-labeling, using a KNN check to filter reliable pseudo-labels, and regional oversampling in a unified process. This combination improves pseudo-label quality and strengthens minority-class representation through selective filtering and in-cluster augmentation. Consequently, the model achieves substantial gains in both precision and recall, validating the proposed Regional Self-Training framework as a practical and effective enhancement for IDS.

Method	**Precision**	**Recall**	**F1-score**	**Accuracy**
Standard-ST [19]	91.6%	93.9%	92.7%	98.4%
FixMatch [36]	93.9%	93.6%	93.8%	92.9%
Our proposed	95.0%	97.8%	96.4%	99.2%

Table 6. Performance comparison on CICIDS2017 (1% labeled)

Overall, the experimental results on both datasets validate the effectiveness of the proposed framework, consistently achieving higher precision, recall, and F1-Score compared to existing semi-supervised approaches, even under extreme label scarcity.

Chapter 5: Conclusion

The primary challenge in intrusion detection under class imbalance is the scarcity of labeled malicious samples. Manual annotation is labor-intensive and does not scale, so methods that automatically expand high-quality labels from limited initial data are urgently needed. To meet this need, we propose a Regional Self-Training framework that combines density-aware clustering with targeted minority-class augmentation.

Malicious traffic often forms high-density "islands" in the feature space, but global confidence–based pseudo-labeling can misassign labels in low-density regions and amplify majority-class bias. In contrast, our approach employs density-aware pseudo-labeling, which partitions unlabeled data into coherent density regions using HDBSCAN In high-purity clusters, pseudo-labels are assigned by majority vote, while in medium-purity clusters, they are assigned by model predictions but further filtered using a KNN check to ensure reliability. We then apply an in-cluster SMOTE step that generates synthetic minority-class examples only within these clusters, preserving local data structure while further balancing the training set.

Our Regional Self-Training framework addresses both issues in prior methods: density-based clustering with KNN check ensures comprehensive yet reliable coverage of the unlabeled space, and in-cluster augmentation enhances minority-class representation without introducing noise. Extensive experiments on UNSW-NB15 and CICIDS2017 show that, using only 1% labeled data, our method improves the overall F1-Score from 93.60% to 96.80% (+3.20 percentage points) on UNSW-NB15 and from 92.70% to 96.40% (+3.70 percentage points) on CICIDS2017, compared to traditional self-training. Minority-class recall also increases from 94.50% to 98.00% (+3.50 percentage points) on UNSW-NB15 and from 93.90% to 97.80% (+3.90 percentage points) on CICIDS2017. By integrating density-aware pseudo-labeling with KNN check and targeted in-cluster oversampling, the proposed framework significantly improves both the quantity and quality of labeled malicious data, enabling more accurate and robust intrusion detection in imbalanced real-world environments.

Chapter 6: Future Work

In future work, we will pursue two main directions to enhance our Regional Self-Training framework.

First, we aim to extend the framework from binary to multi-class attack detection. To achieve this, we will replace the current purity criterion with an entropy-based measure, adjust pseudo-label assignment with class-weighted predictions, and apply in-cluster SMOTE separately to minority attack types. The KNN check will be retained to reduce mislabeling across similar attack families, allowing the method to better distinguish and balance multiple attack categories.

Second, we plan to mitigate concept drift by incrementally reclustering newly arriving data and dynamically adjusting cluster-purity thresholds based on recent performance. Instead of recomputing clusters entirely, the model will update existing clusters with new samples and reorganize regions affected by distributional shifts. A time-based retraining mechanism will also be introduced, gradually replacing older samples with newly labeled and augmented ones. This strategy improves adaptability while maintaining efficiency and reducing the need for manual intervention.

Chapter 7: References

[1] S. Hameed, F. I. Khan, and B. Hameed, "Understanding security requirements and challenges in Internet of Things (IoT): A review," Journal of Computer Networks and Communications, vol. 2019, no. 1, p. 9629381, 2019.

[2] H. Oz, A. Aris, A. Levi, and A. S. Uluagac, "A survey on ransomware: Evolution, taxonomy, and defense solutions," ACM Computing Surveys, vol. 54, no. 11s, pp. 1–37, 2022.

[3] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "Survey on incremental approaches for network anomaly detection," arXiv preprint, arXiv:1211.4493, 2012.

[4] R. Kale, Z. Lu, K. W. Fok, and V. L. Thing, "A hybrid deep learning anomaly detection framework for intrusion detection," in Proc. IEEE Int. Conf. Big Data Security on Cloud (BigDataSecurity), High Performance and Smart Computing (HPSC), and Intelligent Data and Security (IDS), May 2022, pp. 137–142.

[5] S. Kushal, B. Shanmugam, J. Sundaram, and S. Thennadil, "Self-healing hybrid intrusion detection system: An ensemble machine learning approach," Discover Artificial Intelligence, vol. 4, no. 1, p. 28, 2024.

[6] R. Kimanzi, P. Kimanga, D. Cherori, and P. K. Gikunda, "Deep learning algorithms used in intrusion detection systems—A review," arXiv preprint, arXiv:2402.17020, 2024.

[7] N. Farnaaz and M. A. Jabbar, "Random forest modeling for network intrusion detection system," Procedia Computer Science, vol. 89, pp. 213–217, 2016.

[8] J. Jha and L. Ragha, "Intrusion detection system using support vector machine," International Journal of Applied Information Systems, vol. 3, pp. 25–30, 2013.

[9] P. Wu, H. Guo, and N. Moustafa, "Pelican: A deep residual network for network intrusion detection," in Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks Workshops (DSN-W), Jun. 2020, pp. 55–62.

[10] T. Talaei Khoei and N. Kaabouch, "A comparative analysis of supervised and unsupervised models for detecting attacks on the intrusion detection systems," Information, vol. 14, no. 2, p. 103, 2023.

[11] B. Williams and L. Qian, "Semi-supervised learning for intrusion detection in large computer networks," Applied Sciences, vol. 15, no. 11, p. 5930, 2025.

[12] X. Zheng, S. Yang, and X. Wang, "Sf-IDS: An imbalanced semi-supervised learning framework for fine-grained intrusion detection," in Proc. IEEE Int. Conf. Communications (ICC), May 2023, pp. 2988–2993.

[13] C. Wei, K. Sohn, C. Mellina, A. Yuille, and F. Yang, "CREST: A class-rebalancing self-training framework for imbalanced semi-supervised learning," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Jun. 2021, pp. 10857–10866.

[14] R. J. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in Proc. Pacific-Asia Conf. Knowledge Discovery and Data Mining (PAKDD), Apr. 2013, pp. 160–172.

[15] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[16] D. Yarowsky, "Unsupervised word sense disambiguation rivaling supervised methods," in Proc. 33rd Annu. Meeting of the Association for Computational Linguistics (ACL), Jun. 1995, pp. 189–196.

[17] M. R. Amini, V. Feofanov, L. Pauletto, L. Hadjadj, E. Devijver, and Y. Maximov, "Self-training: A survey," Neurocomputing, vol. 616, p. 128904, 2025.

[18] T. K. Das, S. Gangopadhyay, and J. Zhou, "SSIDS: Semi-supervised intrusion detection system by extending the logical analysis of data," arXiv preprint, arXiv:2007.10608, 2020.

[19] D. H. Lee, "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," in Proc. Workshop Challenges in Representation Learning, ICML, Jun. 2013, vol. 3, no. 2, p. 896.

[20] J. Bridle, "Training stochastic model recognition algorithms as networks can lead to maximum mutual information estimation of parameters," Advances in Neural Information Processing Systems (NeurIPS), vol. 2, pp. 211–217, 1989.

[21] Y. Zou, Z. Yu, X. Liu, B. V. K. Kumar, and J. Wang, "Confidence regularized self-training," in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), Oct. 2019, pp. 5982–5991.

[22] M. Ester, H. P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD), Aug. 1996, pp. 226–231.

[23] P. Berba, "Understanding HDBSCAN and density-based clustering," stats, Jan. 2020. [Online]. Available: https://pberba.github.io/stats/2020/01/17/hdbscan/. [Accessed: Aug. 17, 2025].

[24] HDBSCAN developers, "Benchmarking Performance and Scaling of Python Clustering Algorithms," HDBSCAN documentation. [Online]. Available: https://hdbscan.readthedocs.io/en/latest/performance_and_scalability.html. [Accessed: Aug. 17, 2025].

[25] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," Cybersecurity, vol. 2, no. 1, pp. 1–22, 2019.

[26] A. Pinto, L. C. Herrera, Y. Donoso, and J. A. Gutierrez, "Survey on intrusion detection systems based on machine learning techniques for the protection of critical infrastructure," Sensors, vol. 23, no. 5, p. 2415, 2023.

[27] H. Liu and B. Lang, "Machine learning and deep learning methods for intrusion detection

systems: A survey," Applied Sciences, vol. 9, no. 20, p. 4396, 2019.

[28] A. Campazas-Vega, I. S. Crespo-Martínez, Á. M. Guerrero-Higueras, C. Álvarez-Aparicio, V. Matellán, and C. Fernández-Llamas, "Malicious traffic detection on sampled network flow data with novelty-detection-based models," Scientific Reports, vol. 13, no. 1, p. 15446, 2023.

[29] S. K. Wagh and S. R. Kolhe, "Effective intrusion detection system using semi-supervised learning," in Proc. Int. Conf. Data Mining and Intelligent Computing (ICDMIC), Sep. 2014, pp. 1–5.

[30] M. Liu, Q. Yang, W. Wang, and S. Liu, "Semi-supervised encrypted malicious traffic detection based on multimodal traffic characteristics," Sensors, vol. 24, no. 20, p. 6507, 2024.

[31] R. A. R. Ashfaq, X. Z. Wang, J. Z. Huang, H. Abbas, and Y. L. He, "Fuzziness based semi-supervised learning approach for intrusion detection system," Information Sciences, vol. 378, pp. 484–497, 2017.

[32] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in Proc. Military Communications and Information Systems Conf. (MilCIS), Nov. 2015, pp. 1–6.

[33] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in Proc. Int. Conf. Information Systems Security and Privacy (ICISSp), 2018, vol. 1, pp. 108–116.

[34] H. Han, W. Y. Wang, and B. H. Mao, "Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning," in Proc. Int. Conf. Intelligent Computing, Aug. 2005, pp. 879–887.

[35] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in Proc. IEEE Int. Joint Conf. Neural Networks (IJCNN, IEEE World Congress on Computational Intelligence), Jun. 2008, pp. 1322–1328.

[36] K. Sohn, D. Berthelot, N. Carlini, Z. Zhang, H. Zhang, C. A. Raffel, and C. L. Li, "Fixmatch: Simplifying semi-supervised learning with consistency and confidence," Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 596–608, 2020.